

utiliser impérativement les feuilles quadrillées fournies pour les schémas simplifiés

Une réponse non justifiée sera considérée comme fausse

On souhaite comparer deux réalisations de l'architecture Mips-32.

La première réalisation, appelée **Mips**, est la réalisation en 5 étages (**IFC**, **DEC**, **EXE**, **MEM** et **WBK**) étudiée en cours.

La seconde réalisation, appelée **MipsO**, est également une réalisation en 5 étages : **IFC**, **DEC**, **EXE**, **MEM** et **WBK**. Les étages **MipsO** sont identiques à ceux de **Mips**.

L'objectif de cette comparaison est d'étudier l'impact de la mise en place de bypass impliquant les instructions de type *load*. En effet, dans **Mips**, lorsqu'une instruction *load* est suivie par une instruction qui dépend du résultat du *load*, des bypass permettent d'éviter des cycles de gel. Ces bypass récupèrent l'opérande mémoire lu par l'instruction *load* dans son étage **MEM** et l'achemine vers l'instruction qui a besoin de cet opérande.

On souhaite étudier l'impact de la suppression de ces bypass dans la réalisation **MipsO**.

- 1- Si l'on ignore les techniques d'optimisation, un compilateur pour **MipsO** est-il différent d'un compilateur pour **Mips** ?
- 2- Dans une réalisation encore plus simplifiée du processeur, les dépendances de données sur le résultat des instructions *load* ne sont pas gérées par le matériel ni en ce qui concerne les bypass ni pour la génération des cycles de gel.
Cette simplification a-t-elle un impact sur le compilateur ?
- 3- Dans **MipsO**, on supprimant les bypass sur les opérands mémoire, on espère augmenter la fréquence de fonctionnement du processeur. Expliquer pourquoi peut-on obtenir une telle amélioration ?
- 4- Dans **Mips** il existe 8 *bypass* pour limiter la dégradation des performances due aux dépendances de données. Quels sont les *bypass* dans la réalisation **MipsO** ? Justifier votre réponse à l'aide de schémas simplifiés. Pour chaque *bypass* donner un exemple de suite d'instructions qui illustre la nécessité de ce *bypass*.

On souhaite comparer les 2 réalisations sur un programme de traitement d'images en 'niveaux de gris'.

Une image est composée d'un ensemble de pixels organisés sous forme d'un tableau. Pour une image en 'niveaux de gris', chaque pixel est codé sur un octet. La valeur 0 représente la couleur noire, la valeur 255 la couleur blanche. Les valeurs comprises entre 0 et 255 représentent les différentes nuances de gris.

Souvent les fonctions de traitement d'images ne sont pas appliquées sur l'ensemble de l'image mais uniquement sur une partie de celle-ci. Pour cela on utilise un masque. Pour chaque pixel, le masque permet d'indiquer si le pixel subit le traitement ou non. Si le masque vaut 0, le pixel n'est pas modifié. Si le masque vaut 255, on lui applique la fonction de traitement. Le masque de l'image est donc un tableau d'octets de même taille que l'image.

La fonction suivante permet d'obtenir le négatif de l'image sur une partie de l'image.

```

unsigned char *neg_msk (unsigned char *pt_img,
                        unsigned char *pt_msk,
                        unsigned int   size )
{
    unsigned int i = 0 ;

    for (i=0 ; i!=size ; i++)
    {
        if (pt_msk [i] != 0)
            pt_img [i] = 255 - pt_img [i];
    }
    return (pt_img);
}

```

Un compilateur (non pipeline) a produit le code assembleur suivant pour la boucle principale de cette fonction. On suppose qu'à l'entrée de la boucle le registre R4 contient l'adresse du tableau **pt_img**, R5 l'adresse du tableau **pt_msk** et le registre R6 l'adresse de fin du tableau **pt_img** (l'adresse qui suit immédiatement celle de la dernière case du tableau). Le registre R7 contient la valeur 255.

Version 1 :

```

_For :  Lbu      r7 , 0 (r5 )      ; pt_msk[i]
        Bne     r7 , r0 , _Endif
        Lbu     r8 , 0 (r4 )      ; pt_img[i]
        Sub     r8 , r7 , r8      ; 255-pt_img[i]
        Sb      r8 , 0 (r4 )

_Endif: Addiu    r5 , r5 , 1
        Addiu    r4 , r4 , 1
        Bne     r4 , r6 , _For

```

- 5- Modifier le code pour qu'il soit exécutable sur **Mips** et analyser l'exécution de la boucle à l'aide d'un schéma simplifié dans le cas où le branchement échoue (faire figurer les bypass sur le schéma).

Combien de cycles sont-ils nécessaires à l'exécution d'une itération de la boucle dans le cas où le branchement échoue et dans le cas où le branchement réussit ? Calculer le CPI et le CPI-utile en supposant que dans 25% des cas le branchement échoue.

- 6- Modifier le code pour qu'il soit exécutable sur **MipsO** et analyser l'exécution de la boucle à l'aide d'un schéma simplifié dans le cas où le branchement échoue (faire figurer les bypass sur le schéma).

Combien de cycles sont-ils nécessaires à l'exécution d'une itération de la boucle dans le cas où le branchement échoue et dans le cas où le branchement réussit ? Calculer le CPI et le CPI-utile en supposant que dans 25% des cas le branchement échoue.

- 7- Optimiser le code de la boucle pour **MipsO** en modifiant l'ordre des instructions de manière à éviter au maximum les cycles perdus (expliquer votre démarche).

Combien de cycles sont-ils nécessaires dans **MipsO** pour effectuer une itération ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.

Même question pour la réalisation **Mips**.

- 8- Pour améliorer les performances d'exécution, on décide de dérouler une fois le code de la boucle (traitement de 2 éléments à chaque itération). Proposer un code optimisé après le déroulement pour **MipsO**.

Combien de cycles sont-ils nécessaires dans **MipsO** pour **traiter un élément** ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.

Même question pour **Mips**.

Un autre compilateur (non pipeline) a produit une version différente pour le code de la boucle en remarquant que lorsque l'on calcule sur 8 bits : $255 - x = \bar{x}$.

Version 2 :

```
_For :  Lbu      r7 , 0 (r5 )      ; pt_msk[i]
        Lbu      r8 , 0 (r4 )      ; pt_img[i]
        Xor      r8 , r7 , r8
        Sb      r8 , 0 (r4 )

        Addiu    r5 , r5 , 1
        Addiu    r4 , r4 , 1
        Bne      r4 , r6 , _For
```

- 9- Après adaptation du code de la version 2 à **MipsO**, combien de cycles sont-ils nécessaires à l'exécution d'une itération de la boucle ? Calculer le CPI-utile. Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code.

Même question pour la réalisation **Mips**.

- 10- Optimiser le code de la boucle en modifiant l'ordre des instructions pour **MipsO** de manière à éviter au maximum les cycles perdus. Combien de cycles sont-ils nécessaires pour effectuer une itération ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.

Même question pour la réalisation **Mips**.

- 11- Pour améliorer les performances d'exécution, on décide de dérouler une fois le code de la boucle (traitement de 2 éléments à chaque itération). Proposer un code optimisé pour **MipsO** après le déroulement. Combien de cycles sont-ils nécessaires pour **traiter un élément** ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.

Même question pour la réalisation **Mips**.

- 12- Pour chacune des 6 versions du code :

- Version 1 (code original, code réordonné, code déroulé)
- Version 2 (code original, code réordonné, code déroulé)

quel gain en fréquence doit-on obtenir pour que la réalisation **MipsO** soit plus performante que la réalisation **Mips** ?